

Phase 2 Documentation

The Phase 2 Vulnerability Correlation module ingests scan results from Phase 1, translates identified services and version strings into Common Platform Enumeration (CPE 2.3) syntax, and queries the NIST NVD REST API 2.0 to retrieve CVE identifiers and CVSS risk scores.

A. Phase 2 Settings and Core Configuration

- **init.py:** Declares `phase2_correlation` as an importable Python package.
- **correlator.py:** Ingests Phase 1 JSON output (`scan_result.json`), parses service banners, formats CPE strings via `build_cpe_string()`, executes API queries against the NVD database, and stores results.
- **config.py Integration:** Uses `pathlib` to dynamically resolve paths across root directories, ensuring data hand-offs between `data/scanner_results/` and `data/vulnerability_results/` regardless of terminal execution context.
- **Rate Limiting:** Integrates a `rate_delay` mechanism (`time.sleep`) between API calls to comply with public NVD REST API rate limits.
- **CPE Translation Dictionaries:** Uses `vendor_map` and `product_map` dictionaries to translate Nmap service identifiers (e.g., `werkzeug httpd`, `apache httpd`, `nginx`) into official NVD vendor and product equivalents (e.g., `palletsprojects:werkzeug`, `apache:http_server`, `f5:nginx`).

B. Mistakes, Problems Faced, and Implemented Fixes:

This phase was challenging because implement a huge database by an API, get the queries from important services & products, and I realized that we need a directory for store the results and be able to handle it in future phases. Based on this table what I found during the developing and testing:

Mistake / Problem Faced	Root Cause	Implemented Fix
Empty NVD Query Results	Application names in service banners did not match official NVD organizational vendor names (e.g., querying NGINX under <code>nginx</code> instead of its parent company <code>f5</code>).	Vendor & Product Mapping: Implemented <code>vendor_map</code> and <code>product_map</code> translation dictionaries in <code>correlator.py</code> to convert raw service strings to exact NVD vendor/product names.
Strict CPE Query Failures	Initial queries relied on the <code>cpeName</code> parameter, which strictly demands a complete, 13-part exact CPE dictionary match.	Dynamic CPE Matching: Switched API query parameters from <code>cpeName</code> to <code>virtualMatchString</code> , allowing the NVD API to match vulnerabilities dynamically on partial CPE strings.

Cross-Directory Import Errors	Executing scripts from nested subdirectories threw <code>ModuleNotFoundError</code> when attempting to load the root <code>config.py</code> .	Root Path Injection & <code>-m</code> Flag: Dynamically modified <code>sys.path</code> to include the project root and standardized script execution using the Python <code>-m</code> package flag.
--------------------------------------	---	---

C. Testing Results:

On the terminal, run the following command on the terminal:

```
py -3 -m phase2_correlation.correlator --file 127_0_0_1.json
```

If you want to name your results of founded vulnerabilities correlated or other scan from phase one, use this command:

```
python3 correlator.py --file another_scan.json --output another_output.json
```

1.1. Output on CLI:

```
PS C:\Users\ZackU\Desktop\Auto-Pentest> py -3 -m phase2_correlation.correlator --file 127_0_0_1.json
[*] Reading scan results from: C:\Users\ZackU\Desktop\Auto-Pentest\data\scanner_results\127_0_0_1.json
[*] Checking Port 135 | Service: Microsoft Windows RPC
-> Querying CPE: cpe:2.3:a:microsoft windows:microsoft windows:RPC:*:*:*:*:*
[!] NVD API Error HTTP 404:
-> Found 0 CVE(s)
[*] Checking Port 902 | Service: VMware Authentication Daemon 1.10
-> Querying CPE: cpe:2.3:a:vmware authentication daemon:vmware authentication daemon:1.10:*:*:*:*:*
[!] NVD API Error HTTP 404:
-> Found 0 CVE(s)
[*] Checking Port 912 | Service: VMware Authentication Daemon 1.0
-> Querying CPE: cpe:2.3:a:vmware authentication daemon:vmware authentication daemon:1.0:*:*:*:*:*
[!] NVD API Error HTTP 404:
-> Found 0 CVE(s)
[*] Checking Port 5000 | Service: Werkzeug httpd 2.0.1
-> Querying CPE: cpe:2.3:a:palletsprojects:werkzeug:2.0.1:*:*:*:*:*
-> Found 10 CVE(s)
[*] Checking Port 8080 | Service: Apache httpd 2.4.49
-> Querying CPE: cpe:2.3:a:apache:http_server:2.4.49:*:*:*:*:*
-> Found 69 CVE(s)
[+] Correlation complete. Saved output to: C:\Users\ZackU\Desktop\Auto-Pentest\data\vulnerability_results\correlated_vulns.json
```

1.2. Results Stored:

```
{ correlated_vulns.json X
data > vulnerability_results > {} correlated_vulns.json > ...
35
36 "port": 5000,
37 "protocol": "TCP",
38 "state": "open",
39 "service": {
40   "name": "http",
41   "banner": "Werkzeug httpd 2.0.1",
42   "version": "Werkzeug httpd 2.0.1"
43 },
44 "vulnerabilities": [
45   {
46     "cve_id": "CVE-2022-29361",
47     "score": 9.8,
48     "description": "Improper parsing of HTTP requests
49   },
50   {
51     "cve_id": "CVE-2023-46136",
52     "score": 8.0,
53     "description": "Werkzeug is a comprehensive WSGI
54   },
55   {
```